Week 9

Housing Data Cleaning and Transformation

Name: VEENESHWARI R

Roll No:E25DA044

1.Introduction

A real-estate organization wants to study past house sales to understand the features of each property and the factors linked to house prices. The raw data has many columns and some rows that are not useful or not realistic. Before any analysis can be done, the data must be cleaned and prepared.

In this report I used Power Query inside Power BI Desktop to clean the King County House Sales dataset, create new columns, group the houses into size and price categories, and load the final table into the Power BI data model.

Problem Statement

Analyze historical house sales data to understand property characteristics and the factors associated with house prices.

Objectives

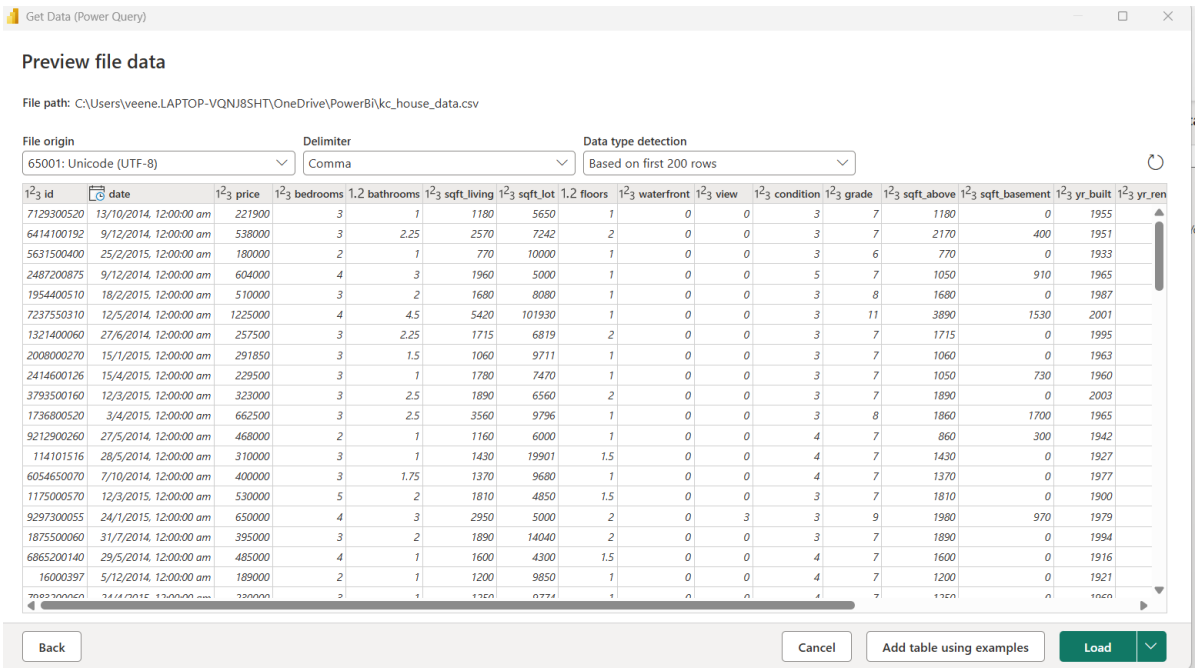
- Import the real estate records into Power Query and clean missing or invalid entries.
- Get date details from the sale date: Sale Year, Sale Month and Property Age at Sale.
- Calculate new columns: Price per SqFt, Total Area and Is Renovated.
- Group the properties into size buckets (Small, Medium, Large, Luxury) and price tiers.
- Remove unrealistic rows with zero bedrooms or zero bathrooms.
- Load a clean and structured real estate data model.

About the Dataset

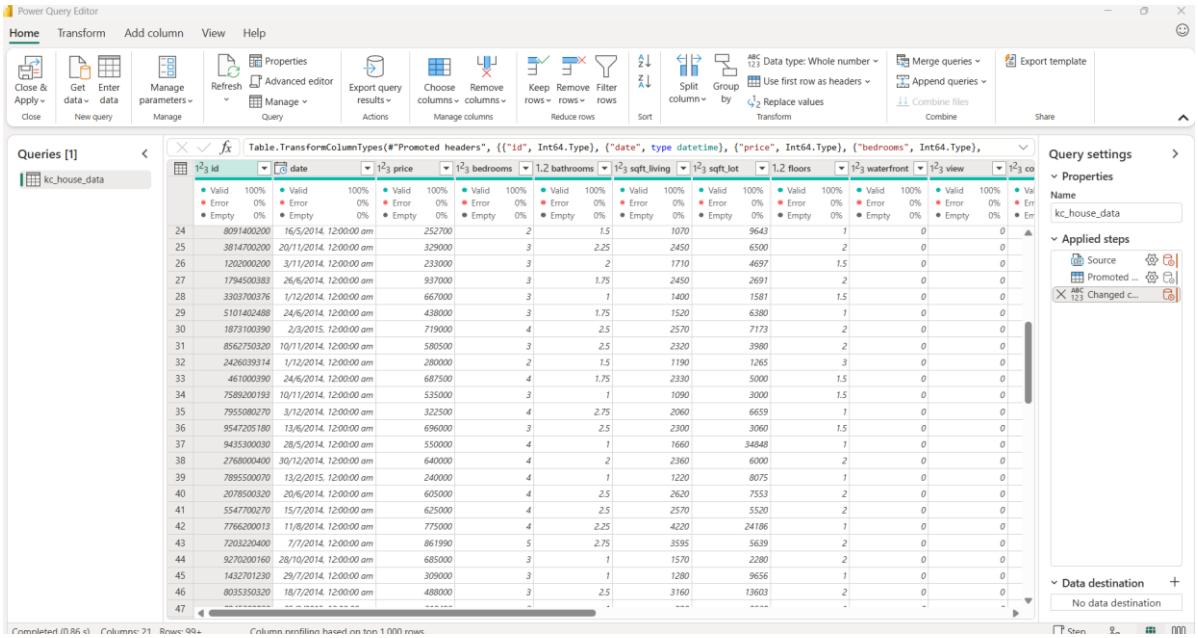
The file kc_house_data.csv has 21,613 rows and 21 columns. It includes the house id, sale date, price, bedrooms, bathrooms, living area, lot area, floors, waterfront, view, condition, grade, year built, year renovated, zipcode, latitude, longitude and the living and lot area of the 15 nearest neighbours.

2. Step 1: Importing the Data

I opened Power BI Desktop and used Home > Get data > Text/CSV to select the file kc_house_data.csv.



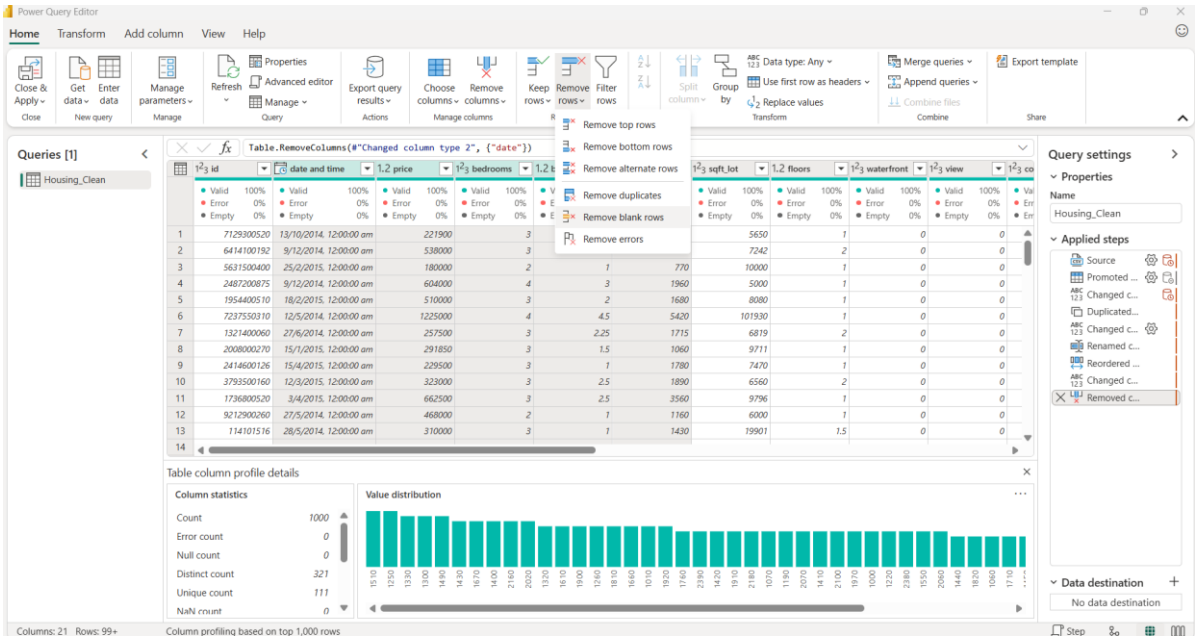
In the Power Query Editor the query appears on the left side. Power Query automatically added the steps Source, Promoted Headers and Changed Column Type, which set the first row as headers and guessed the data type of each column. I later renamed the query to Housing_Clean.



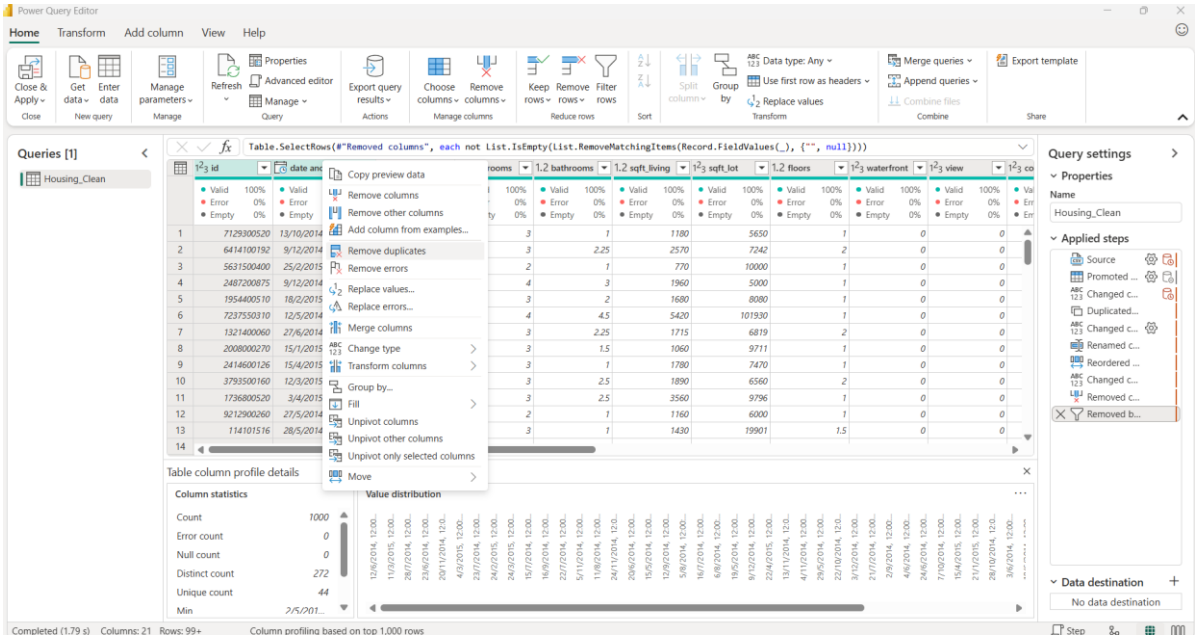
3. Step 2: Cleaning Missing and Invalid Entries

Next I checked the data quality. I turned on Column quality, Column distribution and Column profile from the View tab. These show the percentage of Valid, Error and Empty values for every column. In this dataset all columns showed 0% errors and 0% empty values.

To be safe, I still applied the cleaning options from Home > Remove rows. I used Remove blank rows to delete any row that has no data, and Remove errors to delete any row that contains an error value.

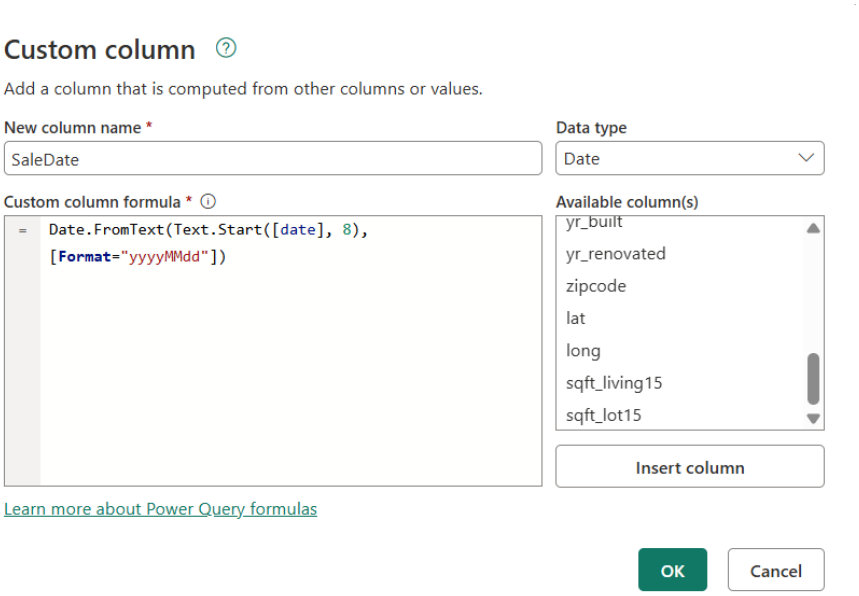


I then selected the id and date columns together, right-clicked the header and chose Remove duplicates. I used both columns together because the same house can be sold more than once on different dates. Removing duplicates by id alone would delete real sales.



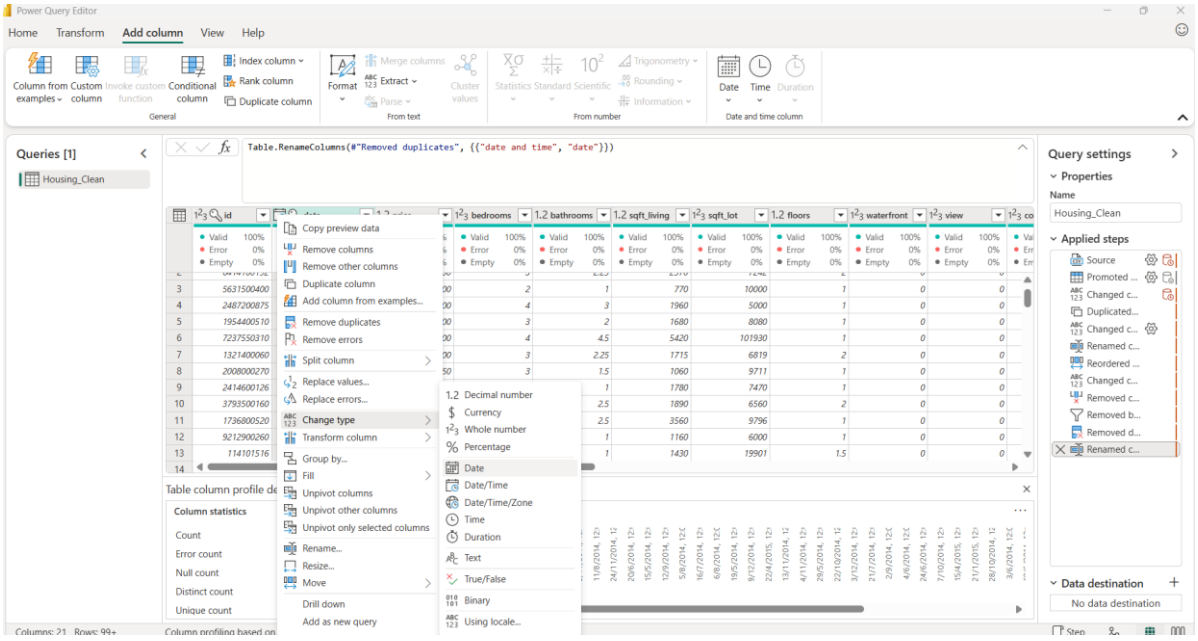
4. Step 3: Parsing the Sale Date

The date column is needed to find the sale year, sale month and property age. At first I tried to create a custom column named SaleDate with a text formula that reads the first 8 characters of the date and converts them to a date.



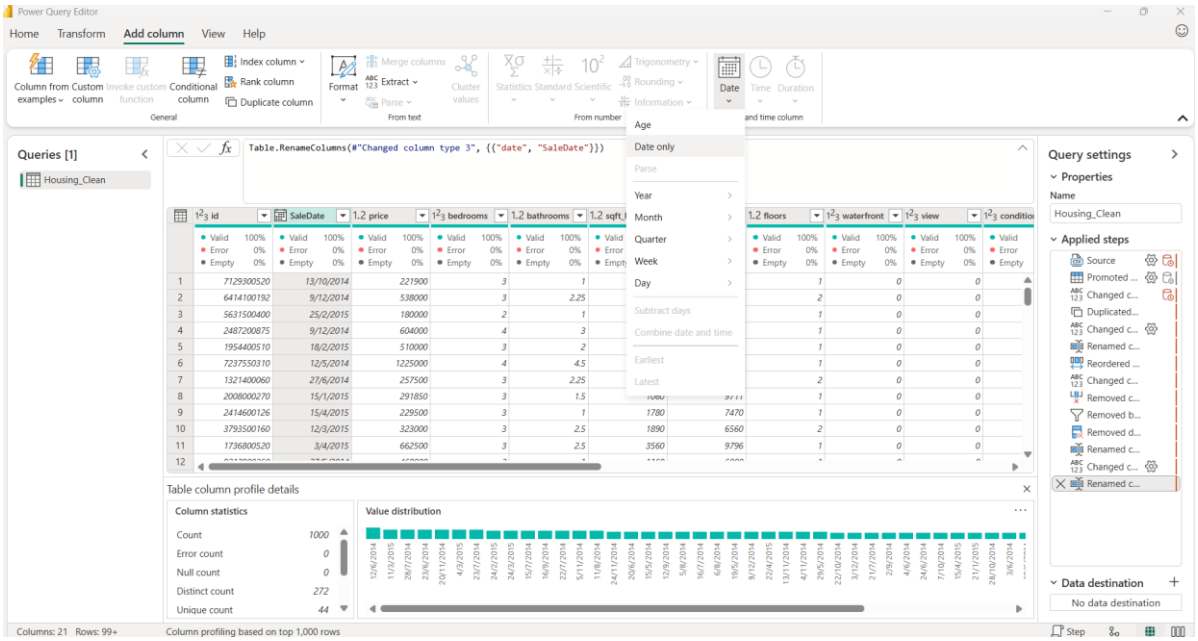
This formula gave errors in every row. The reason is that Power Query had already detected the date column as Date/Time (for example 25/2/2015, 12:00:00 am). A text formula cannot work on a column that is not text.

To fix this, I removed the failed step. I right-clicked the date column, chose Change type and selected Date. This removed the time part and left only the date. I then renamed the column to SaleDate. The column now shows 0% errors.

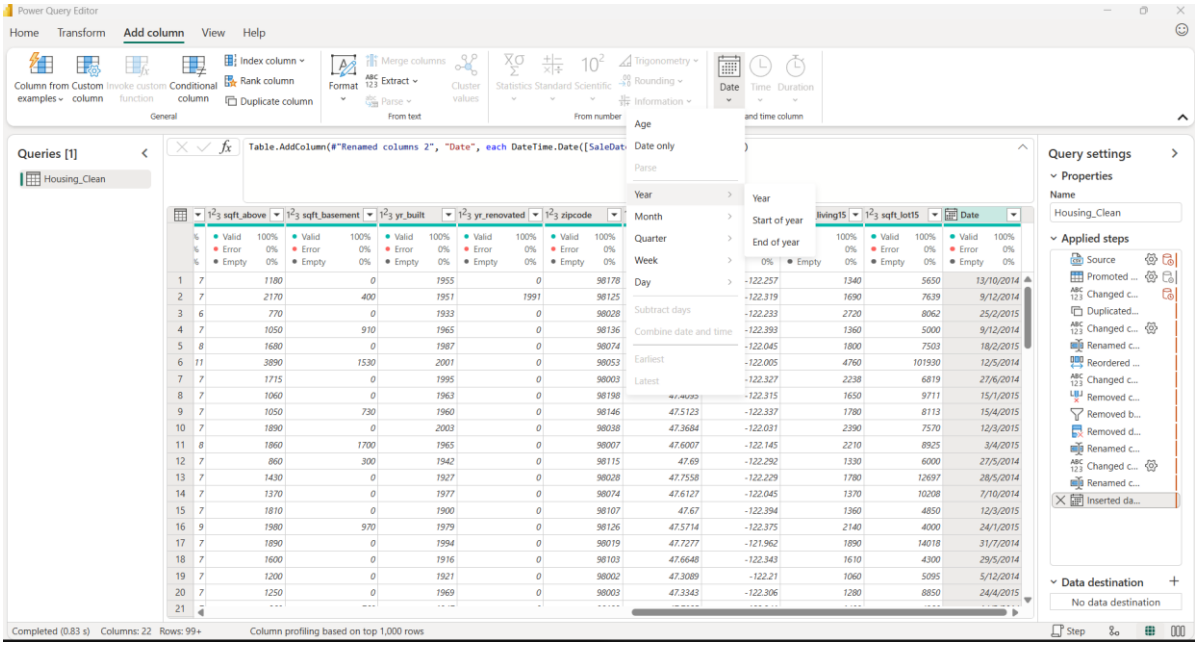


5. Step 4: Sale Year and Sale Month

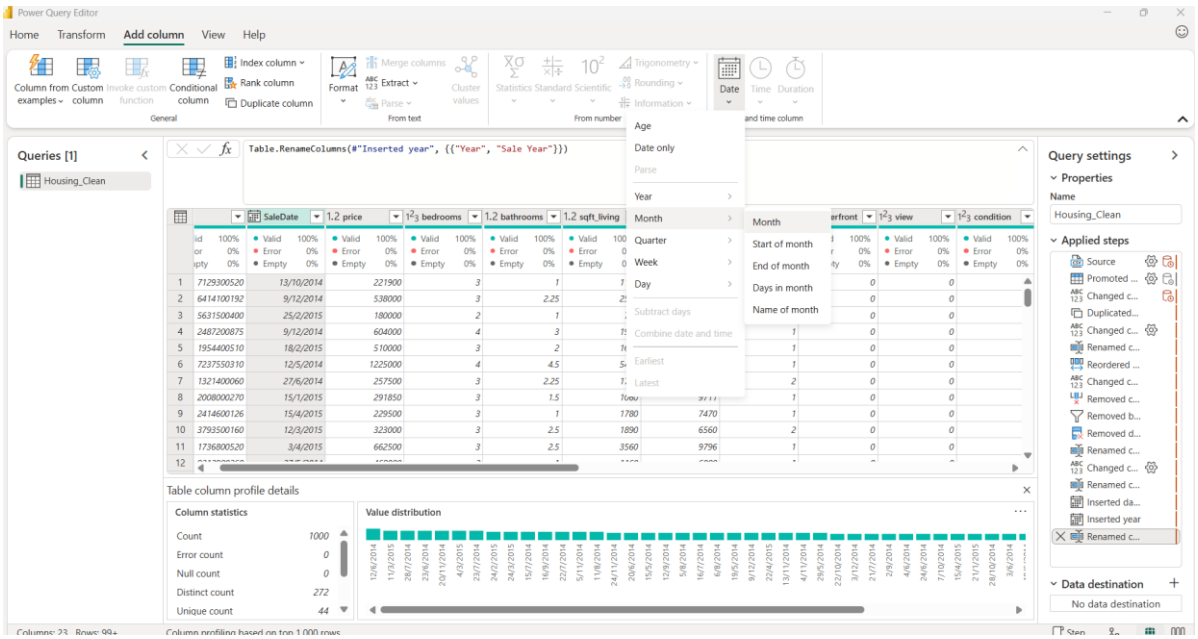
With SaleDate as a proper Date column, I could extract its parts. I selected the SaleDate column, opened the Add column tab and clicked the Date button in the Date and time column group. The menu shows options such as Age, Date only, Year, Month, Quarter, Week and Day.



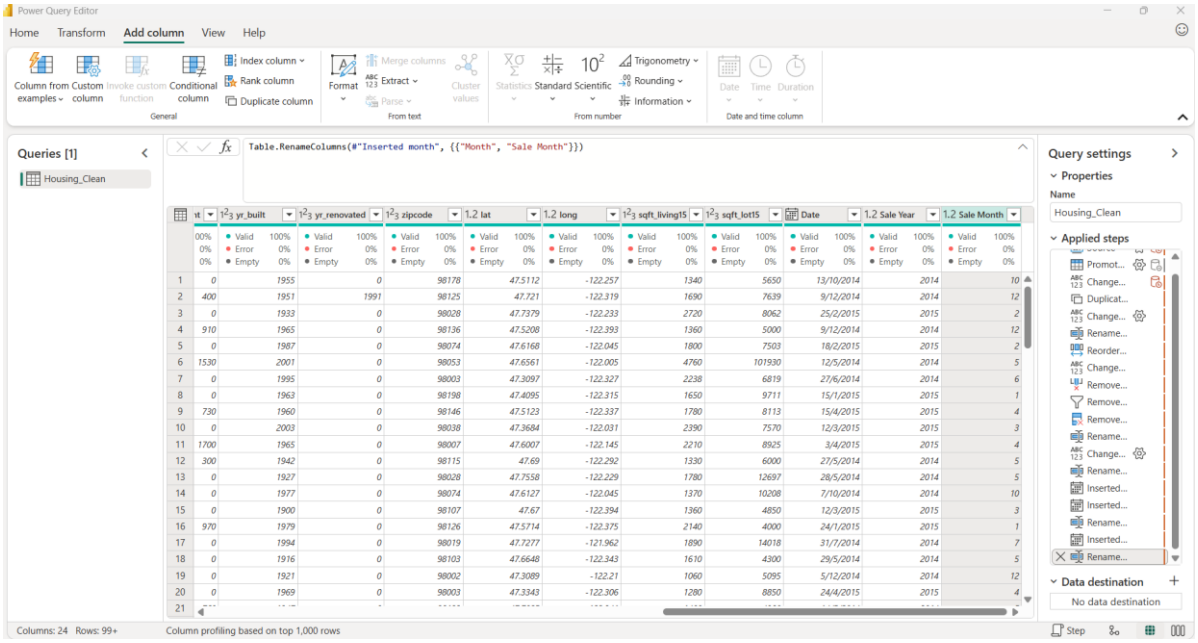
I pointed to Year and clicked Year. This created a new column with the year of sale. I renamed it to Sale Year.



I selected SaleDate again, opened Date > Month and clicked Month. This created a column with the month number from 1 to 12. I renamed it to Sale Month. The same menu also has Name of month, which gives names like January if needed.



After renaming, the table shows Sale Year with values 2014 and 2015, and Sale Month with values from 1 to 12. Both columns show 0% errors.

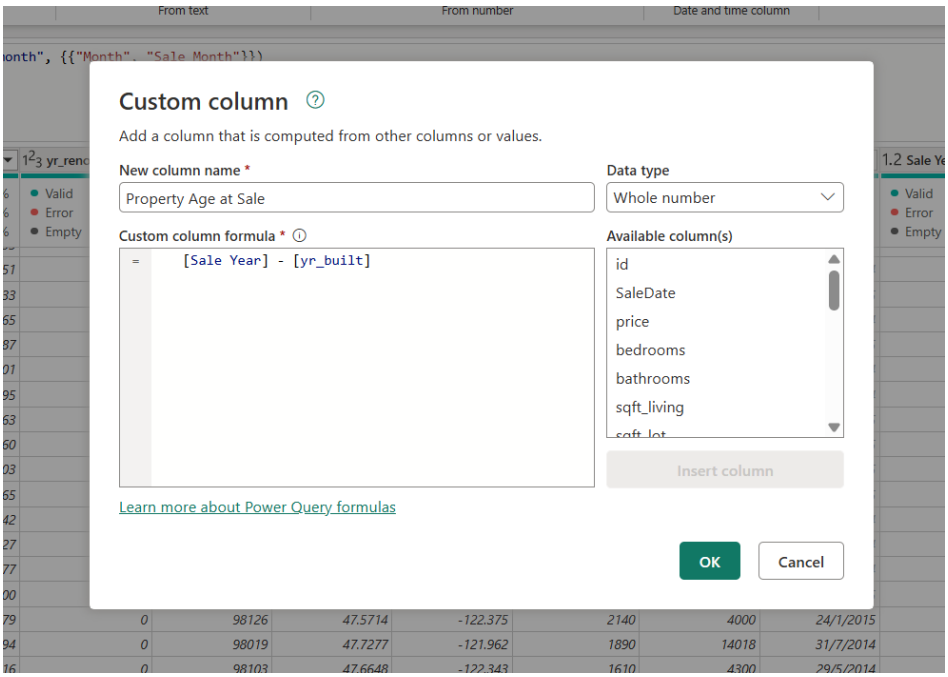


6. Step 5: Creating Derived Columns

I created four new columns using Add column > Custom column. For each one I typed the column name, wrote the formula and chose the data type.

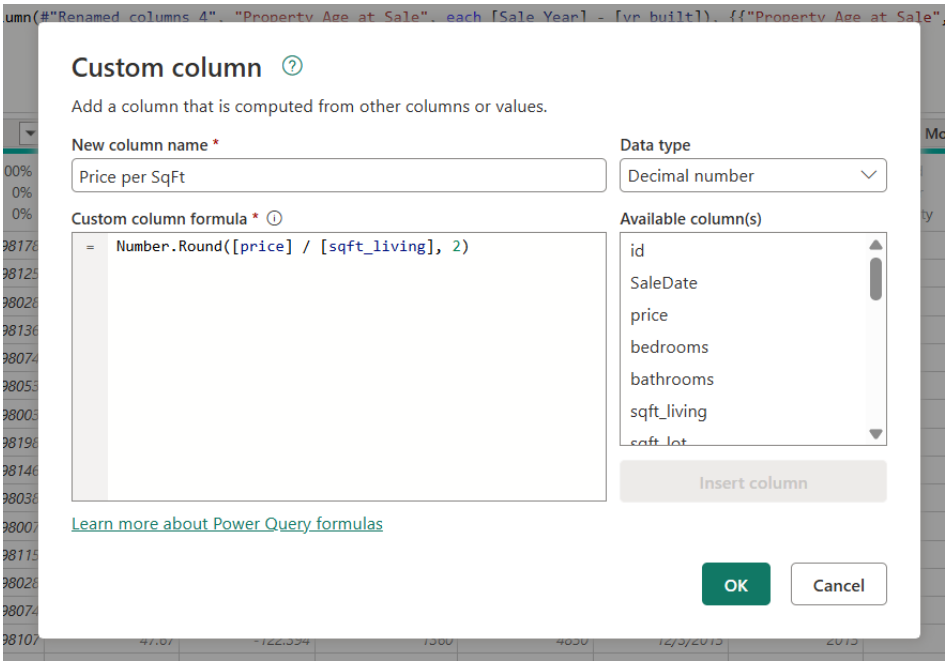
Property Age at Sale

This is the age of the house on the day it was sold. It is the Sale Year minus the year the house was built. Formula: [Sale Year] - [yr_built]. Data type: Whole number.



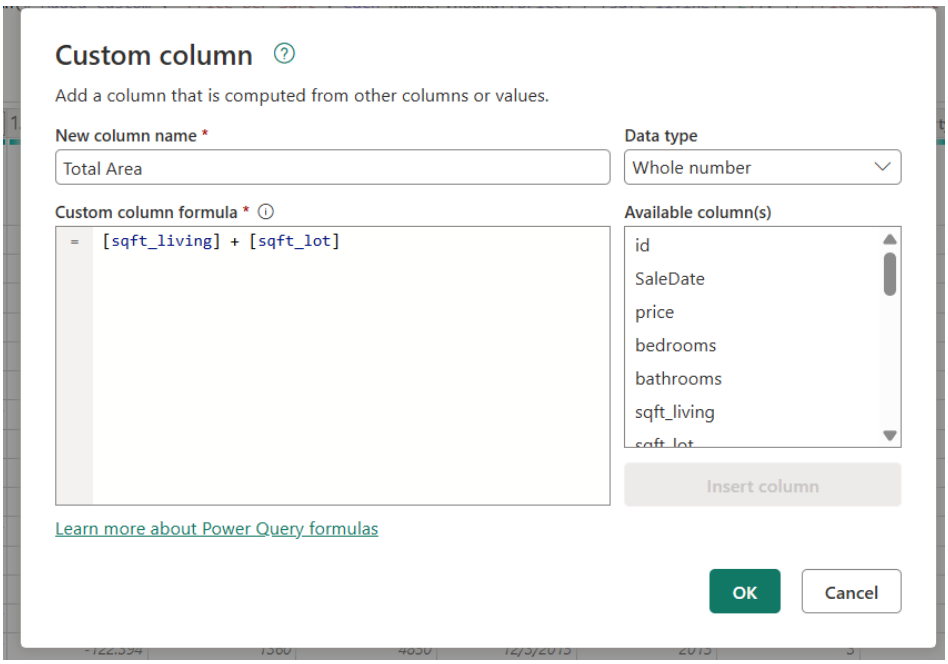
Price per SqFt

This shows how much is paid for each square foot of living space. It is the price divided by the living area, rounded to 2 decimals. Formula: Number.Round([price] / [sqft_living], 2). Data type: Decimal number.



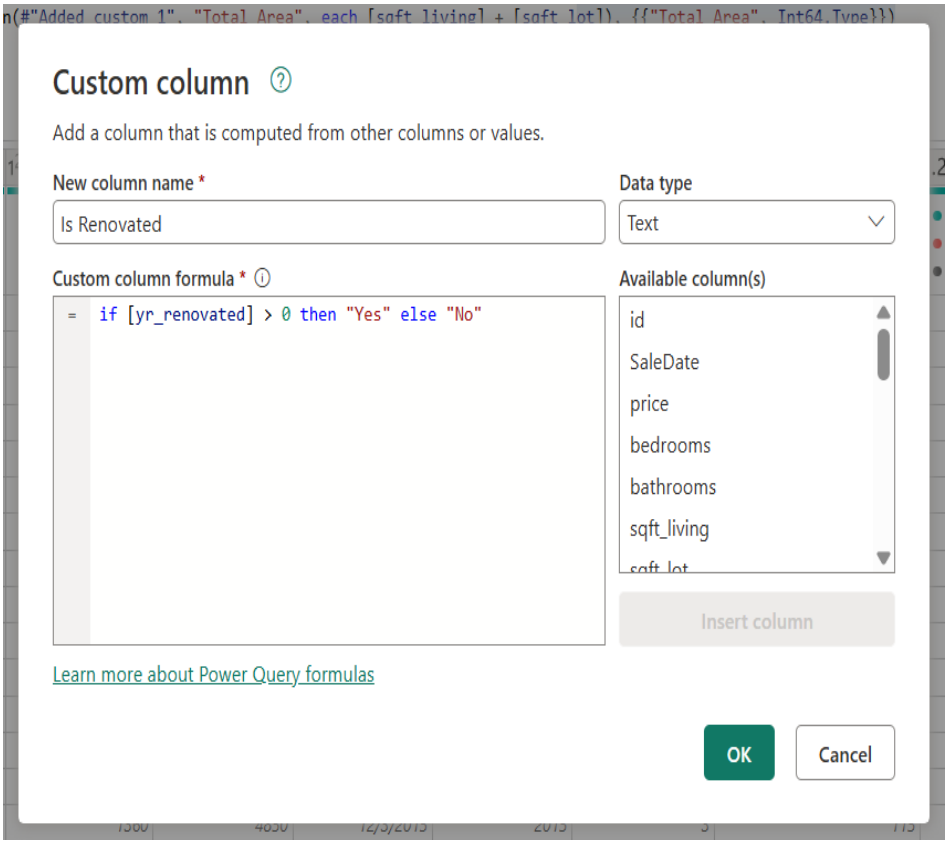
Total Area

I defined Total Area as the living area plus the lot area. Formula: [sqft_living] + [sqft_lot]. Data type: Whole number. This is my assumption, because the dataset has no ready-made total area column.



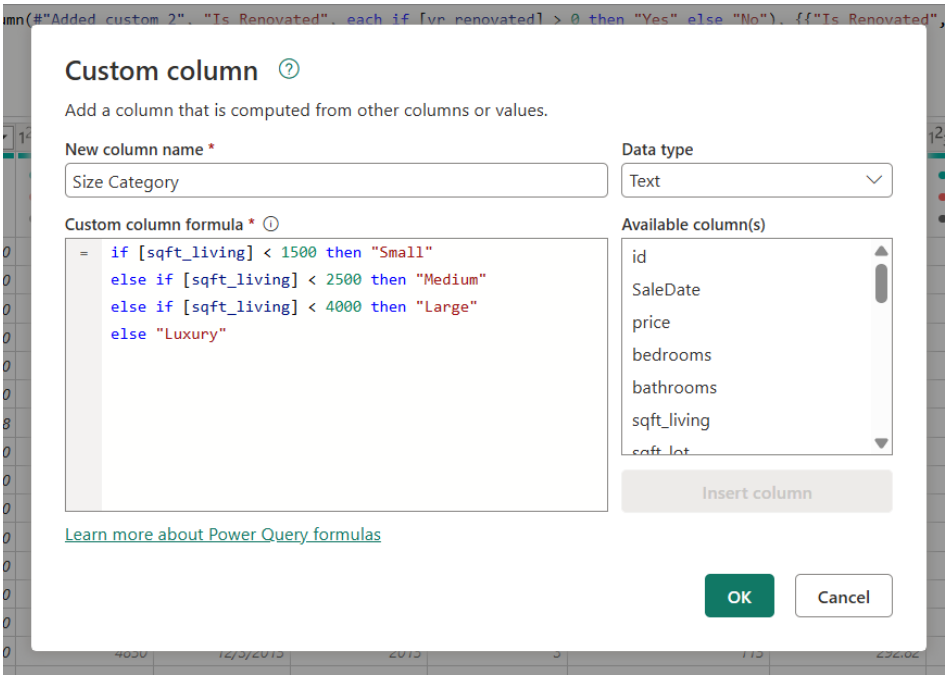
Is Renovated

In this dataset, yr_renovated is 0 when the house was never renovated. So if yr_renovated is greater than 0 the answer is Yes, otherwise No. Formula: if [yr_renovated] > 0 then "Yes" else "No". Data type: Text.

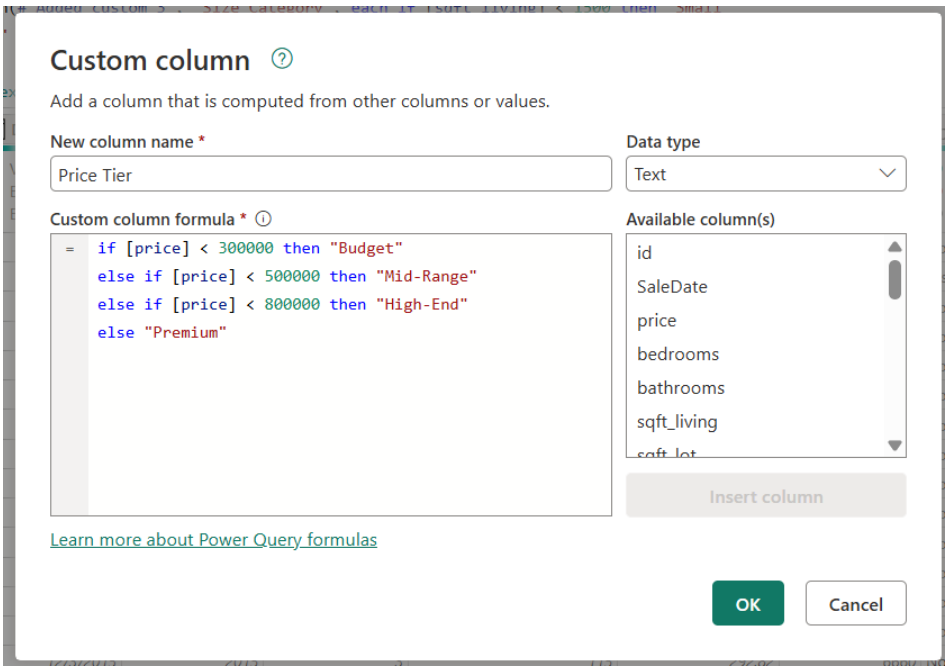


7. Step 6: Size Buckets and Price Tiers

To make the data easier to compare, I grouped the houses into categories using conditional formulas. The Size Category column is based on the living area (sqft_living). It has four groups: Small, Medium, Large and Luxury.



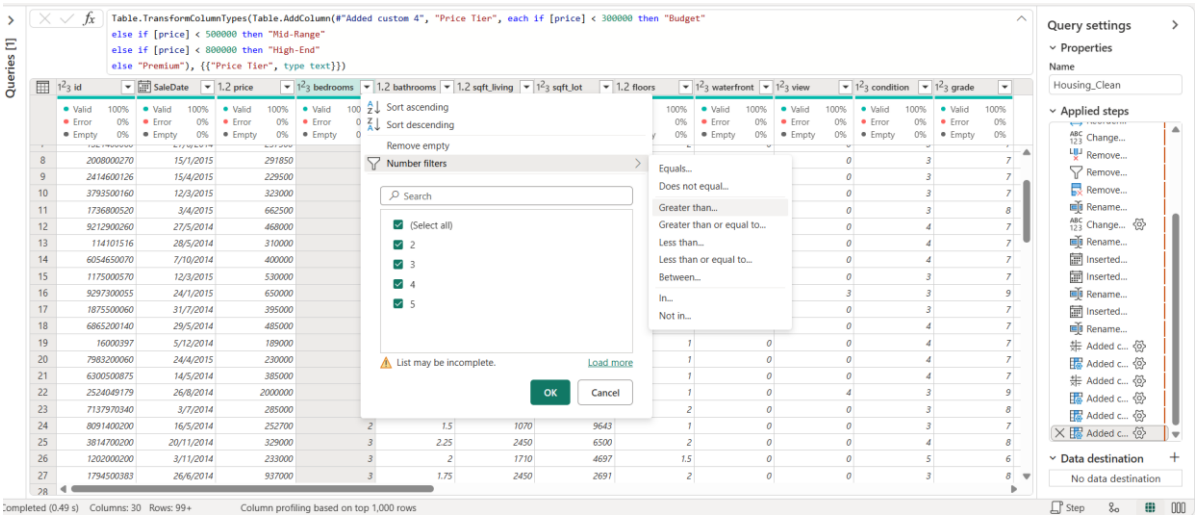
The Price Tier column groups the houses by price into Budget, Mid-Range, High-End and Premium.



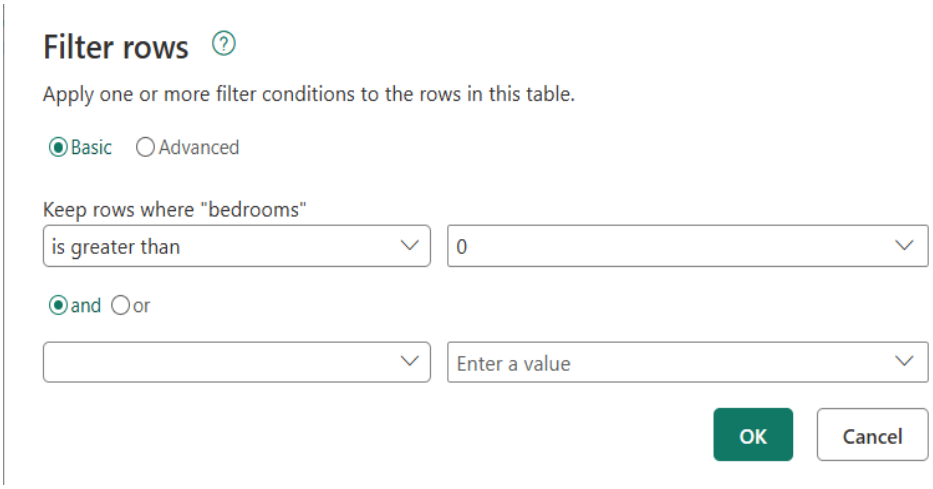
8. Step 7: Filtering Out Unrealistic Rows

A house cannot have zero bedrooms or zero bathrooms for a normal sale, so such rows are treated as invalid data. I removed them using number filters.

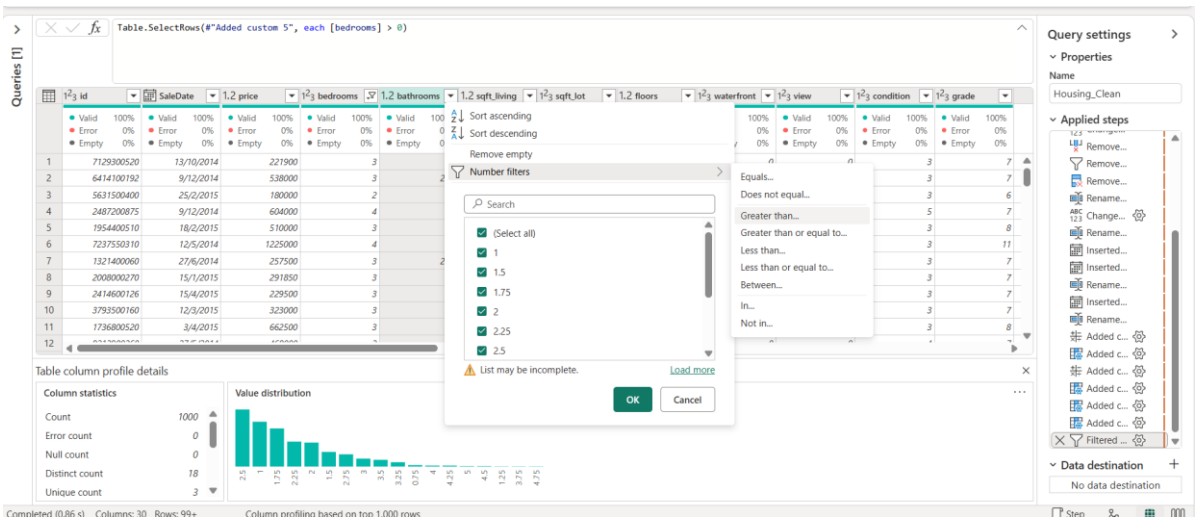
For the bedrooms column, I clicked the drop-down arrow on the header, chose Number Filters and then Greater Than.



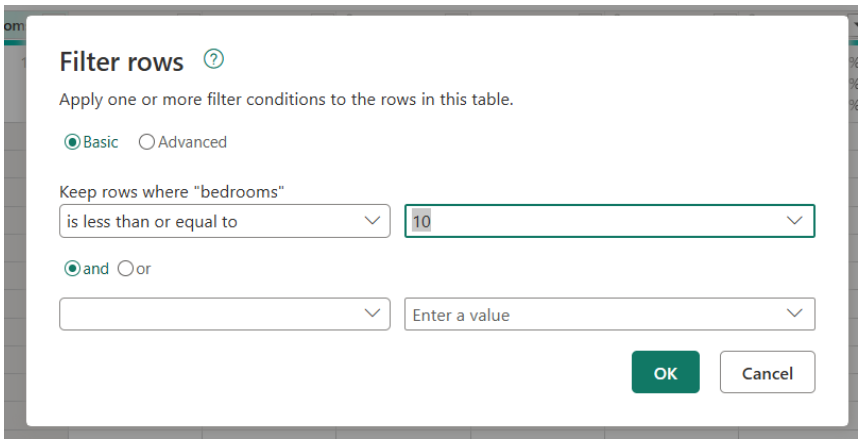
I did the same on the bathrooms column: drop-down arrow, Number Filters, Greater Than, and a value of 0. The Applied Steps list now shows Filtered rows steps.



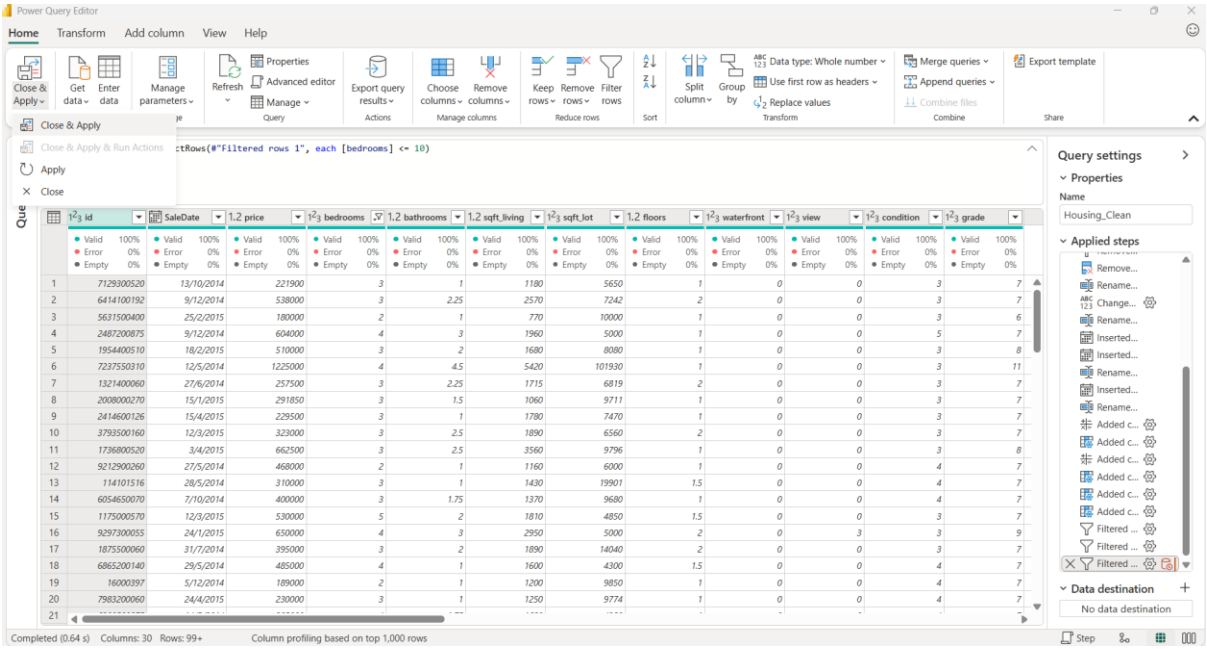
In the Filter rows window I set the condition and clicked OK. The first window below keeps only rows where bedrooms is less than or equal to 10. I added this because one house in the dataset has 33 bedrooms, which is almost certainly a typing mistake.



The second window keeps only rows where bedrooms is greater than 0, which removes houses with zero bedrooms. The same rule was applied to bathrooms.



8. Step 8: close and apply



10. Conclusion

The King County dataset was cleaned and prepared in Power Query. Blank rows and errors were removed, duplicate sales records were handled using id and date together, and the sale date was converted into a proper Date column. From it I created Sale Year, Sale Month and Property Age at Sale. I also calculated Price per SqFt, Total Area and Is Renovated, and grouped the houses into Size Categories and Price Tiers. Rows with zero bedrooms or zero bathrooms, and the 33-bedroom outlier, were filtered out.

The result is a clean table that is ready for analysis, such as comparing price per square foot by year or counting houses by size and price tier.

11. Final Step: Close & Apply

After all cleaning steps were finished, I loaded the data into Power BI. In the Power Query Editor I went to the Home tab and clicked Close & Apply. This closes the Power Query Editor, applies every step in the Applied Steps list, and loads the final Housing_Clean table into the Power BI data model.

What I did

- Clicked Home > Close & Apply in the Power Query Editor.
- Waited until the data finished loading into Power BI Desktop.
- Opened Data view and checked that all new columns are present: SaleDate, Sale Year, Sale Month, Property Age at Sale, Price per SqFt, Total Area, Is Renovated, Size Category and Price Tier.
- Opened Model view to confirm the Housing_Clean table is loaded in the data model.
- Added a Card visual with the count of id in Report view to read the final number of rows.



Figure 21: Close & Apply and the loaded Housing_Clean data model

The structured real estate data model is now ready for analysis and reporting.